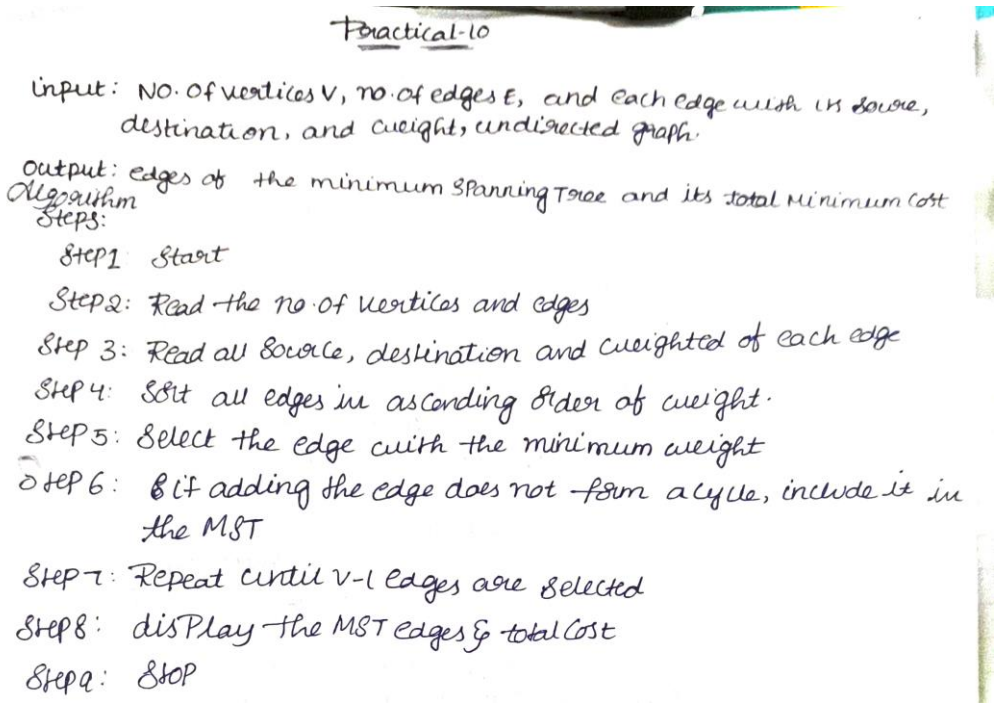
PRACTICAL-10

Aim: To Implementation of Kruskal's algorithm.

Algorithm:



Practical-10

Input: No. of vertices V, no. of edges E, and each edge with its source, destination, and weight, undirected graph.

Output: edges of the minimum Spanning Tree and its total Minimum Cost

Algorithm

Steps:

Step 1 Start

Step 2: Read the no of vertices and edges

Step 3: Read all source, destination and weighted of each edge

Step 4: Sort all edges in asconding order of weight.

Step 5: Select the edge with the minimum weight

Step 6: If adding the edge does not form a cycle, include it in the MST

Step 7: Repeat until V-1 edges are selected

Step 8: display the MST edges & total cost

Step 9: Stop

Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Edge {
   int u, v, w;
};

int parent[100];

int find(int x) {
   if (parent[x] == x)
      return x;

   return parent[x] = find(parent[x]);
}

bool compare(Edge a, Edge b) {
```

```
    return a.w < b.w;
}

int main() {
    int V, E;
    Edge edges[100];

    cout << "Enter number of vertices and edges: ";
    cin >> V >> E;

    cout << "Enter source, destination and weight:\n";

    for (int i = 0; i < E; i++) {
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    }

    for (int i = 0; i < V; i++) {
        parent[i] = i;
    }

    sort(edges, edges + E, compare);

    int cost = 0;
    int count = 0;

    cout << "\nEdges in Minimum Spanning Tree:\n";

    for (int i = 0; i < E && count < V - 1; i++) {
        int a = find(edges[i].u);
        int b = find(edges[i].v);

        if (a != b) {
            cout << edges[i].u << " - "
                 << edges[i].v << " : "
                 << edges[i].w << endl;

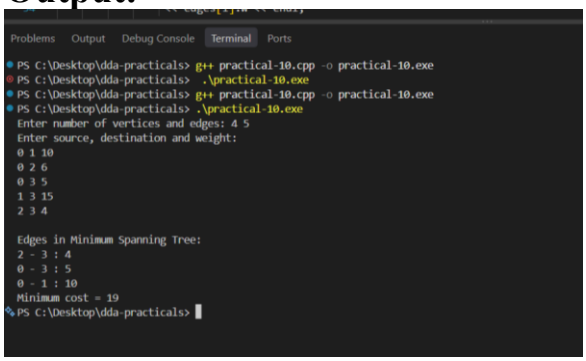
            parent[a] = b;
            cost += edges[i].w;
            count++;
        }
    }

    if (count != V - 1) {
        cout << "MST is not possible because the graph is disconnected.\n";
    } else {
        cout << "Minimum cost = " << cost << endl;
```

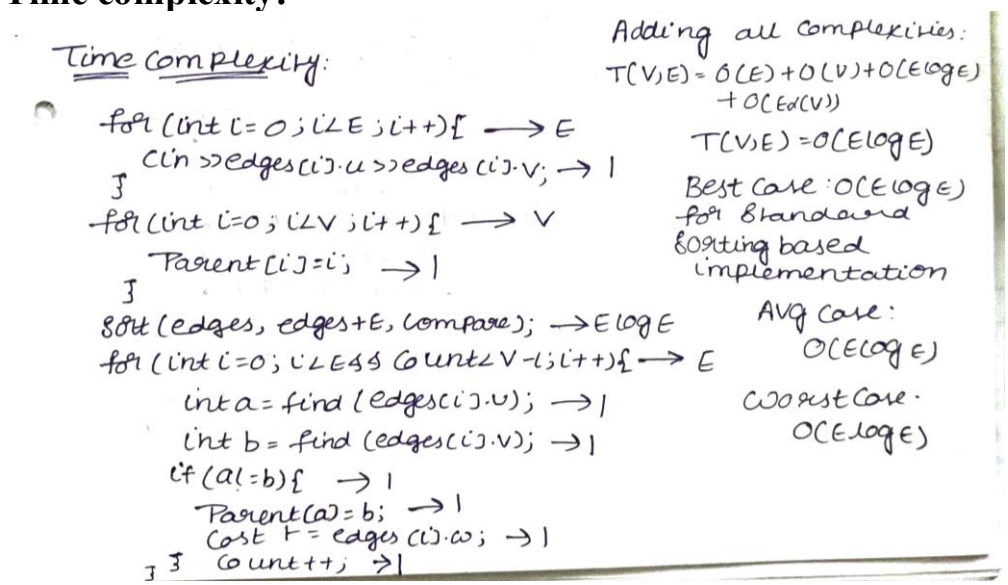
```
    }

    return 0;
}
```

Output:



Time complexity:



Result:

Thus, Kruskal's algorithm was implemented successfully in C++ to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph by selecting edges in increasing order of weight without forming cycles. The minimum total cost was calculated successfully. The time complexity is (O(Elog E)).